File Permissions in Linux

Managing Authorization | Google Cybersecurity Certificate

PROJECT DESCRIPTION

As a security professional supporting a research team, part of my role is ensuring that users are authorized with the correct file permissions. This activity involves examining the existing permissions on the /home/researcher2/projects directory, identifying any mismatches against the organization's authorization policy, and using Linux commands to correct them.

The organization's policy prohibits write access for others on any file, requires that hidden archived files have no write access for anyone, and restricts access to the drafts subdirectory to only the owner. All changes were made using the chmod command in the Linux Bash shell.

CHECK FILE AND DIRECTORY DETAILS

To view all file and directory permissions, including hidden files, I used the ls command with the -la flags from inside the /home/researcher2/projects directory:

```
researcher2@a1b2c3:~$ cd /home/researcher2/projects
```

```
researcher2@a1b2c3:/home/researcher2/projects$ ls -la
```

The -l flag displays a long listing format that includes permission strings, ownership, file size, and modification date. The -a flag shows all files, including hidden files that start with a dot. The output was:

```
total 32 drwxr-xr-x 3 researcher2 research 4096 Jun 19 00:00 . drwxr-xr-x 3
researcher2 research 4096 Jun 19 00:00 .. -rw-rw-rw- 1 researcher2 research  46 Jun
19 00:00 project_k.txt -rw-r----- 1 researcher2 research  46 Jun 19 00:00
project_m.txt -rw-rw-r-- 1 researcher2 research  46 Jun 19 00:00 project_r.txt -rw-
rw-r-- 1 researcher2 research  46 Jun 19 00:00 project_t.txt -rw--w---- 1
researcher2 research  46 Jun 19 00:00 .project_x.txt drwx--x--- 2 researcher2
research 4096 Jun 19 00:00 drafts
```

DESCRIBE THE PERMISSIONS STRING

Taking project_r.txt as an example, its 10-character permission string is: -rw-rw-r--

Here is what each character represents:

```
- rw- rw- r-- | |   |   | | |   |   Other: read only (r--) | |   Group: read and
write (rw-) | User: read and write (rw-) File type: - = regular file (d =
```

```
directory)
```

The first character indicates the file type. A hyphen (-) means it is a regular file; a d means it is a directory. Characters 2-4 represent the user (owner) permissions. Characters 5-7 represent the group permissions. Characters 8-10 represent permissions for others, meaning all users outside the owner and group. An r means read access is granted, a w means write access is granted, an x means execute access is granted, and a hyphen means that permission is not granted.

CHANGE FILE PERMISSIONS

The organization does not allow write access for others on any files. Reviewing the output above, project_k.txt had the permission string -rw-rw-rw-, meaning others had write access. I removed write access for others using chmod:

```
researcher2@a1b2c3:/home/researcher2/projects$ chmod o-w project_k.txt
```

```
researcher2@a1b2c3:/home/researcher2/projects$ ls -la project_k.txt -rw-rw-r-- 1
researcher2 research 46 Jun 19 00:00 project_k.txt
```

The o-w argument tells chmod to remove (-) write (w) access from others (o). After running the command, the permission string changed from -rw-rw-rw- to -rw-rw-r--, confirming that others no longer have write access.

CHANGE FILE PERMISSIONS ON A HIDDEN FILE

Hidden files in Linux begin with a dot (.) and are not shown with a standard ls command. The file .project_x.txt is an archived file that should have no write permissions for anyone, but both the user and group should retain read access. Its current permission string was -rw--w----, meaning the user had read and write, and the group had write only. I corrected this using chmod:

```
researcher2@a1b2c3:/home/researcher2/projects$ chmod u-w,g-w,g+r .project_x.txt
```

```
researcher2@a1b2c3:/home/researcher2/projects$ ls -la .project_x.txt -r--r----- 1
researcher2 research 46 Jun 19 00:00 .project_x.txt
```

The argument u-w removes write from the user, g-w removes write from the group, and g+r adds read for the group. The resulting string -r--r----- confirms that neither the user nor the group can write to this archived file, and both can read it.

CHANGE DIRECTORY PERMISSIONS

The drafts subdirectory belongs to researcher2 and should only be accessible to that user. Its current permission string was drwx--x---, which gave the group execute access, allowing group members to enter the directory. I removed that access using chmod:

```
researcher2@a1b2c3:/home/researcher2/projects$ chmod g-x drafts
```

```
researcher2@a1b2c3:/home/researcher2/projects$ ls -la drwx------ 2 researcher2
research 4096 Jun 19 00:00 drafts
```

The argument g-x removes execute (x) from the group (g). The resulting string drwx------ confirms that only researcher2 can read, write, or navigate into the drafts directory. No group or other access exists.

SUMMARY

In this activity, I used Linux Bash shell commands to audit and correct file permissions in the /home/researcher2/projects directory. Using ls -la, I identified that project_k.txt violated the policy by giving others write access, that the hidden file .project_x.txt had incorrect write permissions for both the user and group, and that the drafts directory was accessible to the group when it should be restricted to the owner only.

I used chmod to fix all three issues: removing write access for others on project_k.txt, stripping write and adding read for the appropriate parties on .project_x.txt, and removing group execute on the drafts directory. These changes brought the file system into alignment with the organization's authorization policy and demonstrate the principle of least privilege in practice.